DOCUMENTO DE REQUISITOS DEL PRODUCTO (PRD): SISTEMA ERP MODULAR ENTERPRISE

1. Visión General del Producto y Estrategia Arquitectónica

El diseño y despliegue de un sistema de Planificación de Recursos Empresariales (ERP) a medida para el segmento corporativo exige un equilibrio entre la flexibilidad funcional y la rigidez de su arquitectura de software. En proyectos de esta magnitud, la selección de la arquitectura base no constituye una preferencia estilística, sino una directiva estratégica que condiciona la viabilidad financiera, la mantenibilidad del código a largo plazo y el escalamiento operativo de la organización. Una plataforma carente de límites de dominio rigurosos degenera rápidamente en sistemas fuertemente acoplados o en arquitecturas prematuramente distribuidas que colapsan bajo el peso de su propia sobrecomplejidad operativa. Por tanto, la estrategia arquitectónica del ERP debe establecer fronteras lógicas e infraestructurales infranqueables, garantizando un rendimiento transaccional óptimo y preservando la integridad de las operaciones de negocio.

Para evitar la ineficiencia de los extremos —el acoplamiento monolítico tradicional (" *Big Ball of Mud* ") y la sobreingeniería temprana de los microservicios distribuidos—, el equipo de ingeniería de Antigravity debe implementar una arquitectura de **Monolito Modular** . En las fases iniciales e intermedias de un ERP, la adopción de microservicios introduce una severa penalización operativa debido a la latencia de red en llamadas interservicios, la complejidad de la sincronización asíncrona de datos y la necesidad de orquestar transacciones distribuidas mediante el patrón *Saga* para alcanzar consistencia eventual. El Monolito Modular consolida la ejecución dentro de un único artefacto y proceso de despliegue, pero impone un aislamiento estricto tanto a nivel de código como de almacenamiento. Esta estrategia permite la ejecución de llamadas intermodales directamente en memoria a través de contratos e interfaces públicas, garantizando transacciones ACID locales y atómicas dentro de una base de datos PostgreSQL unificada.

Dimensión de Análisis,Monolito Tradicional,Monolito Modular,Microservicios
Aislamiento de Código,Bajo (Lógica de negocio y llamadas acopladas),Alto (Módulos estrictos con interfaces públicas explícitas),Total (Repositorios y procesos de ejecución independientes)
Aislamiento de Datos,Base de datos compartida sin fronteras de dominio,Esquemas lógicos separados por módulo en un único motor relacional,Base de datos independiente por cada microservicio
Límite de Transacción,Transacción ACID global única y descontrolada,Transacción ACID local por proceso de ejecución,Consistencia eventual mediante Patrón Saga distribuido
Complejidad Operativa,Mínima,Baja-Media,"Alta (Service mesh , descubrimiento, trazabilidad distribuida)"
Despliegue,Un solo artefacto de código,Un solo artefacto unificado,Múltiples artefactos desplegados independientemente
Evolución Futura,Refactorización compleja y acoplamiento progresivo,Extracción limpia de módulos hacia microservicios independientes,Nativa desde el origen

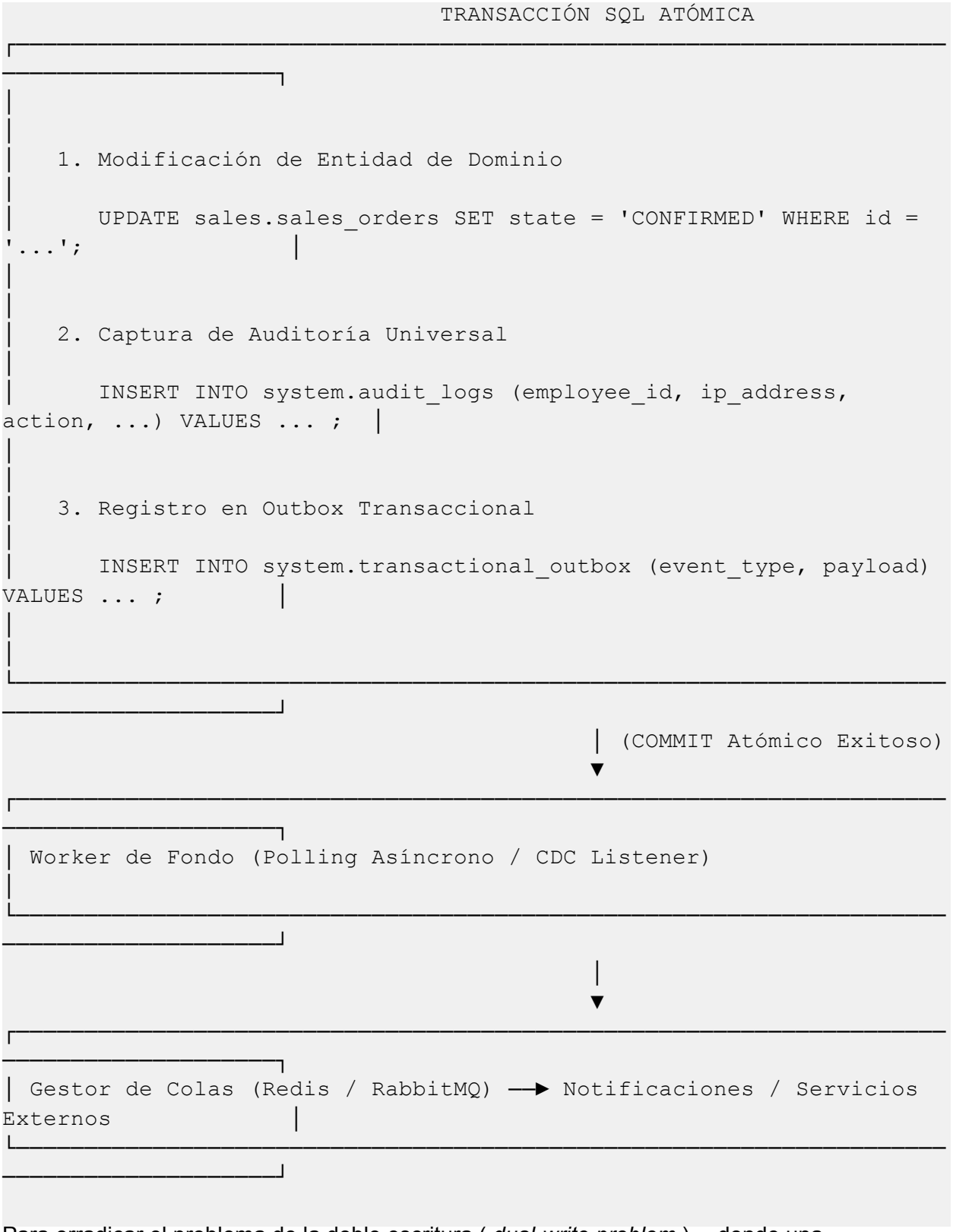
Para garantizar que esta estructura no se degrade con el tiempo, el equipo de Antigravity debe implementar controles automáticos de fronteras en el proceso de compilación y en la tubería de Integración Continua (CI/CD). Queda estrictamente prohibida la importación cruzada de componentes internos entre módulos, así como la ejecución de join SQL directos entre esquemas dentro de los ORM. Cada módulo expondrá exclusivamente un contrato público (API de servicio interna). Antigravity integrará linters de arquitectura en el pipeline de CI/CD (tales como ArchUnit para entornos JVM, import-linter para Python, o reglas de barrera de ESLint para TypeScript) que fallen la compilación de forma automatizada ante cualquier violación de dependencias. Adicionalmente, las migraciones de base de datos se gestionarán mediante un pipeline de cero tiempos de parada (*zero-downtime*) utilizando Flyway o Liquibase, aplicando reglas estrictas de compatibilidad hacia atrás.
Esta severidad en la definición de límites lógicos e infraestructurales constituye el requisito previo para desplegar la capa de seguridad, auditoría e integridad de datos del sistema.

2. Arquitectura del Sistema, Seguridad y Gobernanza de Datos

En un ERP corporativo, la seguridad defensiva en capas, la auditabilidad inalterable y el procesamiento asíncrono de eventos constituyen requisitos no negociables. Dado que la plataforma gestiona transacciones contables, expedientes de personal e inventario físico, la arquitectura debe bloquear cualquier acceso no autorizado en la capa de datos, mantener un historial forense inalterable de cada modificación y asegurar que las notificaciones asíncronas no comprometan la atomicidad de las transacciones SQL relacionales.
El modelo de seguridad coordina el Control de Acceso Basado en Roles (RBAC) en la capa de aplicación con políticas de Seguridad a Nivel de Fila (RLS) en el motor relacional:

- **Control de Acceso Funcional (RBAC):** Define permisos explícitos sobre recursos e interacciones dentro de la tabla permissions, los cuales se agrupan en roles organizacionales dentro de roles. La asignación de accesos se gestiona mediante las tablas asociativas role_permissions (que vincula permisos específicos a cada rol) y employee_roles (que asigna roles a los empleados). Para evaluar reglas de autorización en la capa de servicio, se exige el uso de un motor centralizado como Casbin.
- **Seguridad a Nivel de Fila (PostgreSQL RLS) en Agrupación de Conexiones:** Para garantizar la separación estricta de datos en entornos multisucursal o multipropiedad (*multi-tenant*), se configurarán políticas RLS en PostgreSQL. Debido al uso de agrupadores de conexiones (*connection pools*), el middleware del backend debe ejecutar obligatoriamente SET LOCAL app.current_tenant_id = '...' (o app.current_branch_id) dentro del alcance explícito de cada transacción (BEGIN ... COMMIT). Esto previene la contaminación de contexto (*context leaking*) entre solicitudes ejecutadas secuencialmente sobre un mismo hilo de conexión.

La trazabilidad forense de la plataforma se consolida mediante una **Pista de Auditoría Universal** en la tabla system.audit_logs. Toda transacción de creación, actualización o eliminación lógica insertará de forma atómica un registro inmutable que capturará: ID del empleado ejecutor (employee_id), dirección IP de origen (ip_address), marca de tiempo exacta (action_timestamp), nombre de la tabla impactada (table_name), ID del registro afectado (record_id), tipo de acción (action), y las capturas de estado previo (previous_value) y nuevo (new_value) en formato JSONB.



Para erradicar el problema de la doble escritura (*dual-write problem*) —donde una actualización en la base de datos sucede pero la emisión del mensaje a la cola falla, o viceversa—, el sistema implementará de forma obligatoria el **Patrón Outbox Transaccional** .

Bajo este esquema, cuando una operación de negocio genera un evento (como la confirmación de una orden), el mensaje se inserta en la tabla system.transactional_outbox dentro de la *misma transacción SQL relacional* que modifica la entidad. Un proceso en segundo plano (*worker*) lee asíncronamente la tabla Outbox y publica los mensajes hacia colas externas como Redis o RabbitMQ.
Para sostener cargas de trabajo transaccionales de alto rendimiento en PostgreSQL, Antigravity aplicará las siguientes optimizaciones de motor:

1. **Uso Obligatorio de UUIDv7:** Se prohíbe el uso de UUIDv4 puramente aleatorios para claves primarias. Se exige el estándar UUIDv7, cuyo prefijo temporal monótono garantiza inserciones secuenciales, eliminando la fragmentación de índices B-Tree en tablas de alta frecuencia de escritura.
2. **Índices Parciales:** Se diseñarán índices condicionales sobre estados activos o pendientes (ejemplo: CREATE INDEX idx_orders_pending ON sales.sales_orders (created_at) WHERE state = 'PENDING';).
3. **Carga Anticipada (*Eager Loading*):** La capa ORM debe configurar explícitamente estrategias de *eager loading* y *join prefetching* para prevenir el problema de consultas $N+1$ en la lectura de colecciones relacionales.

Estas directivas de gobernanza y control en la capa de persistencia se complementan con los patrones de modelado inmutable para las entidades de dominio.

3. Patrones Universales de Modelado de Datos e Inmutabilidad

En la arquitectura de este ERP, queda estrictamente prohibida la edición destructiva directa de datos históricos mediante sentencias UPDATE descontroladas. Alterar registros financieros u operativos destruye la auditabilidad, invalida los informes contables e introduce condiciones de carrera (*race conditions*) críticas en entornos concurrentes. La persistenia del dominio se basará en cuatro patrones de inmutabilidad relacional.

Inventario por Doble Entrada

El módulo de inventario aplica el principio contable de partida doble: el stock nunca se incrementa, decrementa o sobrescribe directamente en valores escalares. Todo movimiento físico se registra como un asiento inmutable en el libro mayor de inventario (inventory.stock_moves), identificando una ubicación de origen (source_place_id) y una de destino (dest_place_id).
Las ubicaciones se definen en inventory.stock_places bajo tres tipologías:

- **Ubicaciones Físicas (INTERNAL):** Almacenes, pasillos y estantes bajo control directo de la organización.
- **Ubicaciones de Socios (VENDOR , CUSTOMER):** Nodos virtuales que representan proveedores (origen en compras) y clientes (destino en ventas).
- **Ubicaciones Virtuales (SCRAP , PRODUCTION):** Nodos de ajuste utilizados para mermas, pérdidas, inventarios iniciales o consumo en procesos de fabricación.

El stock disponible no se almacena como una columna editable; se calcula mediante la evaluación analítica de los movimientos:
$$\text{Stock Disponible} = \sum \text{Movimientos de Entrada (Inbound)} - \sum \text{Movimientos de Salida (Outbound)}$$

Para resolver colisiones en escenarios de alta concurrencia durante la reserva de inventario, Antigravity implementará bloqueo pesimista a nivel de fila (SELECT ... FOR UPDATE) sobre las entidades de control de producto/ubicación o el uso de bloqueos ad hoc (*PostgreSQL Advisory Locks*).

Registro Histórico con Fechas de Efectividad (Effective-Dating)

En los dominios de Recursos Humanos y Nómina, los atributos contractuales (compensation_records, asignaciones de puesto job_assignments y retenciones fiscales) evolucionan en el tiempo. La modificación *in-situ* de un registro de salario borra el estado histórico e imposibilita la ejecución de reliquidaciones exactas o auditorías retrospectivas.
Se impone el patrón de *Effective-Dating* : ninguna fila contractual se actualiza sobre sí misma. Todo cambio genera una nueva fila delimitada por los campos effective_from y effective_to. El motor de nómina consulta la fotografía exacta de los registros vigentes en la fecha de liquidación para congelar los recibos de pago.

Estructura Recurso-Jerárquica (BOM)

La definición de productos terminados en el módulo de producción exige estructurar listas de materiales (BOM) compuestas por subensamblajes y materias primas. Este dominio se modela mediante un grafo recursivo en mrp.bom_structures, vinculando un producto padre (parent_product_id) con sus componentes hijos (child_product_id) y su factor de cantidad (qty_ratio). La explosión completa de materiales para el cálculo de requerimientos (MRP) se efectúa mediante Expresiones de Tabla Comunes (CTE) recursivas en SQL, parametrizadas dinámicamente por la cantidad requerida del lote (:order_qty):

```
-- Explosión Recursiva de Materiales (BOM) con Parametrización de
Lote
WITH RECURSIVE bom_explosion AS (
    SELECT
        child_product_id,
        (qty_ratio * :order_qty) AS required_qty,
        1 AS level
    FROM mrp.bom_structures
    WHERE parent_product_id = :target_product_id

    UNION ALL

    SELECT
        b.child_product_id,
        (e.required_qty * b.qty_ratio) AS required_qty,
        e.level + 1
    FROM mrp.bom_structures b
    INNER JOIN bom_explosion e ON b.parent_product_id =
e.child_product_id
)
SELECT
```

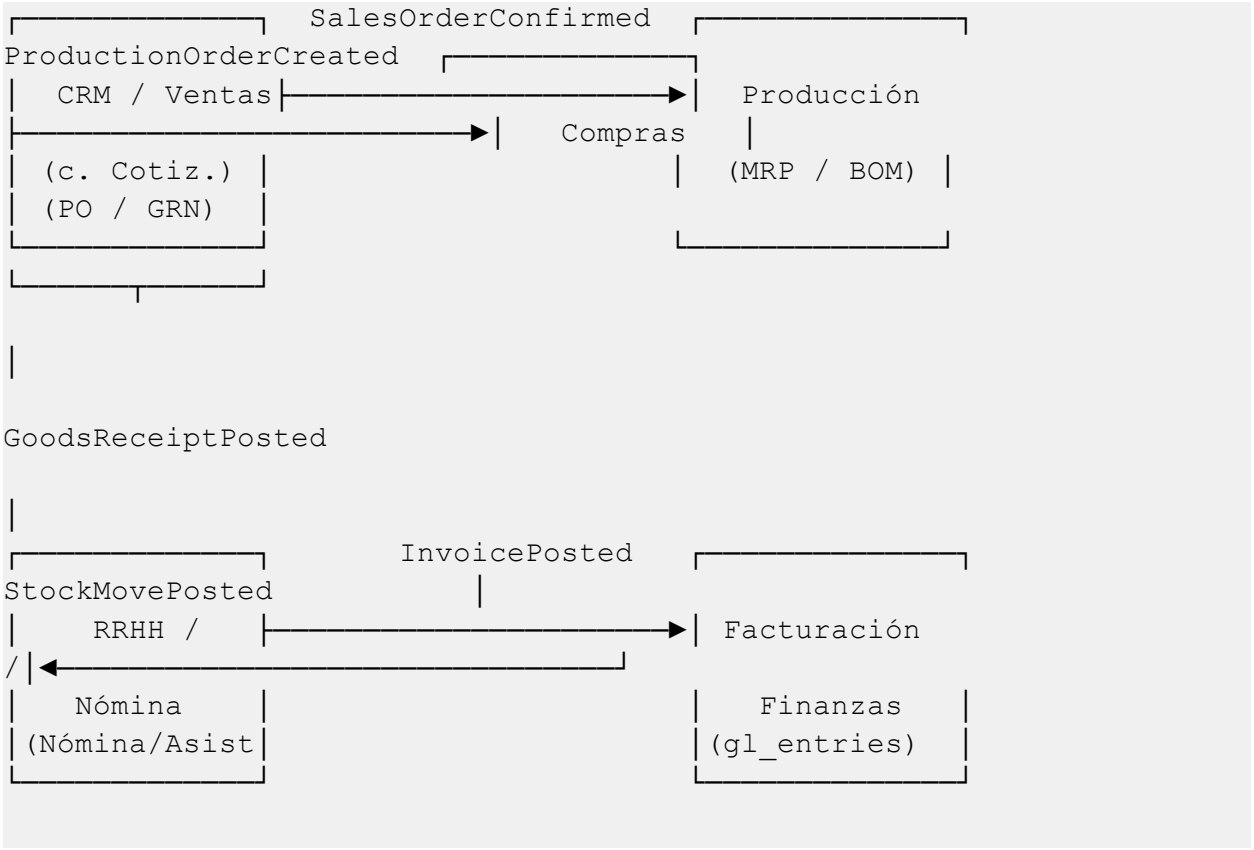
```
    child_product_id,
    SUM(required_qty) AS total_required_qty
FROM bom_explosion
GROUP BY child_product_id;
```

Máquinas de Estado Finitos Deterministas (FSM)

Los documentos transaccionales clave (Cotizaciones, Órdenes de Compra, Órdenes de Fabricación, Facturas) evolucionan exclusivamente a través de estados deterministas (ejemplo: DRAFT $\rightarrow$ CONFIRMED $\rightarrow$ APPROVED $\rightarrow$ POSTED $\rightarrow$ CANCELLED). Las transiciones están controladas por componentes centralizados en el backend que impiden saltos ilegales de estado y emiten eventos atómicos hacia el outbox transaccional.
Estos patrones deterministas de persistencia definen la base técnica para la especificación funcional de los módulos del ERP.

4. Especificación Funcional Módulo por Módulo para el PRD

El ERP abarca ocho dominios operativos interconectados. Ningún módulo operará como una isla de información aislada; la salida transaccional de cada dominio actúa como la entrada operativa del siguiente.



4.1 CRM y Ventas

El submódulo de CRM administra la conversión de prospectos (crm_leads) a través de un embudo de ventas por etapas. Al ganar una oportunidad, el sistema la transforma en una cotización. El módulo de Ventas gestiona las cotizaciones y las convierte en órdenes de venta (sales_orders y sales_order_lines). Valida automáticamente el límite de crédito del cliente, aplica reglas de precios y, al pasar al estado CONFIRMED, emite el evento SalesOrderConfirmed para desencadenar la reserva de inventario o la orden de producción.

4.2 Compras (Procurement)

Administra el ciclo de abastecimiento desde las Solicitudes de Cotización (RFQ) hasta la emisión de Órdenes de Compra (purchase_orders). Incorpora un motor de reordenamiento automático basado en stock mínimo/máximo que genera órdenes borrador al detectar escasez. Asimismo, gestiona la Recepción de Bienes (GRN - *Goods Receipt Note*) para verificar las entregas del proveedor antes de habilitar la entrada de facturas.

4.3 Inventario y Almacén (WMS)

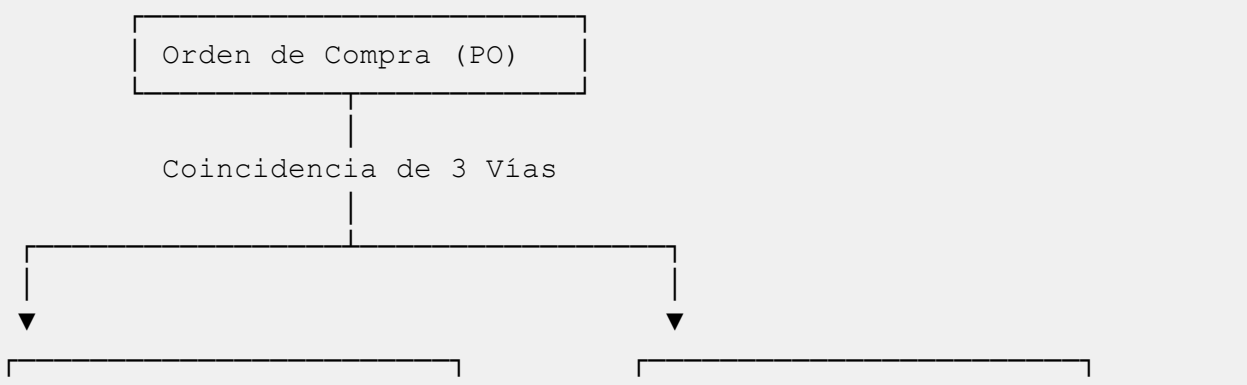
Proporciona control multi-almacén y multi-ubicación (stock_places). Garantiza la trazabilidad por lotes y caducidad aplicando estrategias FIFO (*First In, First Out*) o FEFO (*First Expired, First Out*). Mantiene la valoración continua de stock por Coste Medio Ponderado (WAC) o Coste Estándar, y gestiona los recuentos cíclicos asentando las desviaciones contra ubicaciones virtuales de ajuste.

4.4 Producción (MRP)

Modela la transformación de materias primas mediante Órdenes de Fabricación (production_orders), hojas de ruta (*routings*) y centros de trabajo. Al iniciar una orden, ejecuta la explosión de materiales sobre bom_structures, instruye la reserva de insumos y procesa su consumo mediante movimientos de doble entrada. Al finalizar, registra la entrada del producto terminado y absorbe los costes directos de mano de obra y materiales.

4.5 Facturación y Finanzas

Emite facturas de venta y procesa facturas de proveedores (invoices). La aprobación de facturas de compra exige la verificación del algoritmo de **Coincidencia de 3 Vías** (*3-Way Matching*), validando la coherencia entre la Orden de Compra (purchase_orders), la Recepción de Bienes (GRN) y la Factura.





Al aprobar cualquier factura o movimiento monetario, se efectúa un apunte atómico en el Libro Mayor General (gl_entries), garantizando el balance de la partida doble:
$$\sum \text{Debe} = \sum \text{Haber}$$

4.6 Recursos Humanos, Control de Asistencia y Nómina

El módulo de Recursos Humanos gestiona la estructura de la empresa (companies, branches, departments, job_titles, employment_types), las asignaciones de puesto (job_assignments) y los expedientes del personal (employees), aplicando *Effective-Dating* sobre cambios de puesto y salario (compensation_records).
La asistencia se integra con hardware biométrico registrado en attendance_devices. Las marcas crudas en device_logs se procesan hacia attendance_records para evaluar marcajes, retrasos y turnos (work_shifts). Se gestionan solicitudes de corrección (attendance_corrections), horas extra (overtime_requests, overtime_approvals) y ausencias (leave_requests, leave_approvals), afectando el libro mayor de permisos (leave_ledger).
El motor de nómina calcula los salarios en periodos de pago (pay_schedules, pay_periods, pay_runs). Consolida horas trabajadas (time_sheets, time_sheet_details), aplica deducciones (deduction_types, employee_deductions), retenciones fiscales (tax_jurisdictions, tax_withholding_elections) y genera las líneas de los recibos (paychecks, paycheck_lines). Tras la aprobación del pay_run, los recibos quedan congelados, se procesan los depósitos directos (direct_deposits, vinculados a bank_accounts) y se emite la transacción contable hacia la Contabilidad General.
Esta especificación funcional se consolida en el diseño físico de esquemas y tablas relacionales en PostgreSQL.

5. Esquema de Base de Datos y Modelo Relacional en PostgreSQL

El modelo relacional se implementa en PostgreSQL organizando las tablas en esquemas lógicos por módulo funcional. Para asegurar la integridad referencial y prevenir la orfandad de datos, todas las claves foráneas deben declarar la restricción ON DELETE RESTRICT.
Esquema PG,Tabla,Campos Clave y Tipos de Datos,Propósito y Reglas de Integridad
crm,crm_leads,"id (UUIDv7 PK), employee_id (UUIDv7 FK), stage_id (INT), expected_revenue (DECIMAL)",Oportunidades comerciales. Restricción CHECK para montos no negativos.
sales,sales_orders,"id (UUIDv7 PK), customer_id (UUIDv7 FK), state (VARCHAR), total_amount (DECIMAL)",Encabezado de orden de venta. Estado controlado por FSM.
sales,sales_order_lines,"id (UUIDv7 PK), sales_order_id (UUIDv7 FK), product_id (UUIDv7 FK), qty (DECIMAL), unit_price (DECIMAL)",Detalle de líneas de venta. Clave foránea restrictiva hacia producto.
purchase,purchase_orders,"id (UUIDv7 PK), supplier_id (UUIDv7 FK), state (VARCHAR), order_date (DATE)",Órdenes a proveedores. Requiere Coincidencia de 3 Vías.
inventory,stock_places,"id (UUIDv7 PK), name (VARCHAR), type (VARCHAR: INTERNAL, VENDOR, CUSTOMER, SCRAP, PRODUCTION)",Ubicaciones físicas y virtuales para el inventario por doble entrada.

inventory,stock_moves,"id (UUIDv7 PK), product_id (UUIDv7 FK), source_place_id (UUIDv7 FK), dest_place_id (UUIDv7 FK), qty (DECIMAL)",Libro mayor (ledger) inmutable de inventario. Prohibido el UPDATE directo.
mrp,bom_structures,"id (UUIDv7 PK), parent_product_id (UUIDv7 FK), child_product_id (UUIDv7 FK), qty_ratio (DECIMAL)",Grafo recursivo para listas de materiales. Procesado mediante CTE.
mrp,production_orders,"id (UUIDv7 PK), product_id (UUIDv7 FK), bom_id (UUIDv7 FK), qty (DECIMAL), state (VARCHAR)",Orden de transformación de insumos en producto terminado.
account,invoices,"id (UUIDv7 PK), partner_id (UUIDv7 FK), type (VARCHAR), state (VARCHAR), total_amount (DECIMAL)",Facturación fiscal. Asienta transacciones atómicas en el Libro Mayor.
account,gl_entries,"id (UUIDv7 PK), account_id (UUIDv7 FK), debit (DECIMAL), credit (DECIMAL), posting_date (DATE)",Apunte en el Libro Mayor General. Regla: $\sum \text{Debe} = \sum \text{Haber}$.
hr,employees,"id (UUIDv7 PK), department_id (UUIDv7 FK), job_title_id (UUIDv7 FK), email (VARCHAR UNIQUE)",Expediente central del trabajador. FK restrictiva hacia departamentos y cargos.
hr,job_assignments,"id (UUIDv7 PK), employee_id (UUIDv7 FK), department_id (UUIDv7 FK), job_title_id (UUIDv7 FK), effective_from (DATE), effective_to (DATE)",Asignaciones de puesto históricas con Effective-Dating .
hr,compensation_records,"id (UUIDv7 PK), employee_id (UUIDv7 FK), base_salary (DECIMAL), effective_from (DATE), effective_to (DATE)",Histórico de sueldos con Effective-Dating . Inmutable tras liquidar.
hr,attendance_records,"id (UUIDv7 PK), employee_id (UUIDv7 FK), work_date (DATE), check_in (TIMESTAMP), check_out (TIMESTAMP)",Registros procesados de asistencia a partir de marcajes biométricos.
hr,pay_runs,"id (UUIDv7 PK), pay_period_id (UUIDv7 FK), status (VARCHAR), approved_by (UUIDv7 FK)",Ejecución de nómina. Congela recibos de pago al aprobarse.
system,audit_logs,"id (UUIDv7 PK), employee_id (UUIDv7 FK), ip_address (VARCHAR), action_timestamp (TIMESTAMP), table_name (VARCHAR), record_id (UUIDv7), action (VARCHAR), previous_value (JSONB), new_value (JSONB)",Pista de auditoría universal inalterable del sistema.

Convenciones de Interacción vía REST API

La integración entre capas y servicios externos respetará los estándares REST API:

- **Recursos Plurales e Identificadores UUID:** Rutas basadas en sustantivos en plural identificadas mediante cadenas UUIDv7 (/api/v1/employees, /api/v1/attendance-records, /api/v1/leave-requests).
- **Filtrado, Paginación y Ordenamiento:** Uso estandarizado de parámetros de consulta (?department_id=018f3a2b-8c1e-7f1a-9a3b-123456789abc&page=1&per_page=20&sort=-created_at).
- **Payloads JSON Estructurados:** Los datos enviados en peticiones POST o PATCH deben utilizar tipos UUID estrictos para las referencias a claves foráneas.

```
// POST /api/v1/employees (Payload de Creación de Empleado)
```

```
{
  "department_id": "018f3a2b-8c1e-7f1a-9a3b-123456789abc",
  "job_title_id": "018f3a2c-9d2f-8a2b-0b4c-987654321def",
  "employment_type_id": "018f3a2d-0e3a-9b3c-1c5d-456789123ghi",
  "first_name": "Roberto",
  "last_name": "Gómez",
  "email": "roberto.gomez@empresa.com",
  "phone": "+525551234567",
  "hire_date": "2026-03-01"
}
```

Esta estructura relacional y de interfaces sostiene la ejecución de flujos transaccionales complejos entre los módulos del ERP.

6. Flujos Operativos Intermodales e Integración End-to-End

El rendimiento y la integridad de la plataforma se aseguran mediante la orquestación atómica y asíncrona de sus flujos de trabajo intermodales. Se especifica la secuencia técnica paso a paso que ejecuta el ERP durante el ciclo operativo completo: **Venta $\rightarrow$ Fabricación $\rightarrow$ Compra $\rightarrow$ Recepción $\rightarrow$ Entrega $\rightarrow$ Facturación / Contabilidad** .

1. **Confirmación Comercial:** Un ejecutivo de ventas confirma una cotización. La orden en sales.sales_orders cambia al estado CONFIRMED. La máquina de estados inserta el evento SalesOrderConfirmed en system.transactional_outbox dentro de la misma transacción SQL.
2. **Verificación de Inventario e Instrucción de Fabricación:** El módulo de inventario procesa el evento y evalúa la disponibilidad del producto sumando las entradas y salidas en inventory.stock_moves. Al detectar escasez, el motor de reglas emite una instrucción al módulo de producción (mrp), generando una Orden de Fabricación en estado DRAFT en mrp.production_orders (evento ProductionOrderCreated).
3. **Explosión MRP y Compra de Componentes Faltantes:** El módulo de producción ejecuta una consulta CTE recursiva parametrizada (:order_qty) sobre mrp.bom_structures para desglosar la materia prima requerida. Al calcular el déficit de componentes, el módulo de compras emite una Orden de Compra en estado borrador (purchase.purchase_orders) asignada al proveedor predeterminado.
4. **Recepción de Materias Primas por Doble Entrada:** El proveedor entrega los insumos. El almacenero registra la nota de recepción (GRN). El módulo de inventario inserta un registro inmutable en inventory.stock_moves, trasladando los artículos desde la ubicación virtual VENDOR_LOCATION hacia la ubicación física INTERNAL_WAREHOUSE (evento GoodsReceiptPosted).
5. **Ejecución de Producción y Consumo de Insumos:** La orden en mrp.production_orders pasa al estado IN_PROGRESS. Se trasladan los insumos desde INTERNAL_WAREHOUSE hacia la ubicación virtual PRODUCTION_LOCATION mediante stock_moves. Al finalizar el ensamblaje, la orden pasa a COMPLETED y se

registra la entrada del producto terminado moviéndolo desde PRODUCTION_LOCATION hacia INTERNAL_WAREHOUSE.

6. **Despacho, Facturación y Asiento Doble Atómico:** Se procesa el despacho. El producto terminado se desplaza en stock_moves desde INTERNAL_WAREHOUSE hacia CUSTOMER_LOCATION (StockMovePosted). Se genera la factura en account.invoices y, al confirmarla (InvoicePosted), el sistema ejecuta una transacción SQL atómica que registra el ingreso, la cuenta por cobrar y el coste de ventas en account.gl_entries, validando de forma estricta que $\sum \text{Debe} = \sum \text{Haber}$.

Esta secuencia intermodal define la base operativa del plan de desarrollo del equipo de Antigravity.

7. Hoja de Ruta de Implementación para Antigravity

La hoja de ruta de desarrollo para el equipo de Antigravity está diseñada para mitigar riesgos técnicos, priorizando la construcción de las barreras lógicas, las herramientas de migración y los motores de inmutabilidad antes de desplegar la lógica operativa y las interfaces de usuario.

```
SEMANAS   01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20
21 22 23 24
FASE 1    [=========] (Esquemas / CI/CD / Linters / Migraciones
Flyway)
FASE 2                [=========] (Núcleo Inmutable: stock_moves +
gl_entries + Outbox)
FASE 3
[==================================] (Módulos Operativos S1-S4)
FASE 4
[=========] (Pruebas Carga / RLS / Migration Audit)
```

- **Fase 1: Especificación de Esquemas, Migraciones Cero-Downtime y Contratos de Interfaz (Semanas 1-4):**
- Diseño e implementación de esquemas PostgreSQL por módulo con restricciones ON DELETE RESTRICT.
- Definición estricta de contratos de API REST para cada dominio utilizando identificadores UUIDv7.
- Instalación y configuración obligatoria de linters de arquitectura (ArchUnit, import-linter, ESLint boundary rules) en el pipeline de CI/CD para bloquear compilaciones con dependencias o consultas cross-schema no autorizadas.
- Despliegue del sistema de migraciones *zero-downtime* (Flyway/Liquibase) con reglas de compatibilidad hacia atrás.
- **Fase 2: Construcción del Núcleo de Inmutabilidad y Gobernanza (Semanas 5-8):**
- Desarrollo del libro mayor de inventario por doble entrada (inventory.stock_moves) e integración de bloqueos pesimistas (SELECT ... FOR UPDATE) para gestión de concurrencia.
- Desarrollo del motor contable del Libro Mayor General (account.gl_entries) con validación de partida doble.

- Implementación de la Pista de Auditoría Universal (system.audit_logs) y del patrón Outbox Transaccional (system.transactional_outbox).
- **Fase 3: Implementación Iterativa de Módulos Operativos (Semanas 9-20):**
- *Sprint 1 (Semanas 9-11):* Módulos de CRM, Ventas, Compras e Inventario WMS básico.
- *Sprint 2 (Semanas 12-14):* Módulo de Facturación, Coincidencia de 3 Vías (*3-Way Matching*) y Finanzas.
- *Sprint 3 (Semanas 15-17):* Producción (MRP), Grafos recursivos BOM parametrizados y Hojas de Ruta.
- *Sprint 4 (Semanas 18-20):* Recursos Humanos (*Effective-Dating* en job_assignments y compensation_records), Asistencia Biométrica y Motor de Nómina Inmutable.
- **Fase 4: Verificación Arquitectónica, Pruebas de Estrés y Despliegue (Semanas 21-24):**
- Pruebas de estrés y concurrencia sobre escrituras en stock_moves y gl_entries.
- Validación de la inyección de contexto de sesión RLS (SET LOCAL app.current_tenant_id) sobre la agrupación de conexiones (*connection pooling*).
- Auditoría de planes de ejecución en PostgreSQL, verificación del uso de UUIDv7, índices parciales y ausencia de consultas $N+1$ en la capa ORM.
- Ejecución de pruebas de migración de esquema sin interrupción del servicio (*zero-downtime verification*).

Con la entrega de este Documento de Requisitos del Producto (PRD), se traslada formalmente la especificación técnica al equipo de Antigravity para dar inicio inmediato a la ejecución del proyecto.